RIGOR App: A Demo System for Retrieval-Augmented Generation of Ontologies from Relational Databases

Nadeen Fathallah[1,*], Mojtaba Nayyeri[1] and Steffen Staab[1,2]

[1]Analytic Computing, Institute for Artificial Intelligence, University of Stuttgart, Stuttgart, Germany
[2]University of Southampton, Southampton, UK

Abstract

Relational databases hold much of the world's operational data, but their schemas often fail to convey the semantics of tables, columns, and relationships. RIGOR generates semantically rich OWL2DL ontologies from relational schemas, yet it exists only as a research pipeline that a database owner cannot run in a user-friendly manner. We present the *RIGOR App*, a Web system that makes the method usable: upload a schema file, optionally provide documentation, vocabularies, and a seed ontology as context, and download an enriched ontology with labels, hierarchy, alignments, and provenance. A built-in validation stack repairs syntax errors, logical inconsistencies, and Critical and Important OOPS! findings, while presenting repairs that could change the domain meaning for user confirmation. The *RIGOR App* is open source under the MIT license, with code and the Northwind sample at https://github.com/NadeenAhmad/RIGOR_APP. Two full screencasts (a shorter, unnarrated run and a longer, narrated run) are available at https://www.youtube.com/playlist?list=PLV3vgCoFRQI4.

Keywords

Relational databases, Ontology learning, Large language models, Retrieval-augmented generation.

1. Introduction

Most operational data in hospitals, companies, and public institutions lives in relational databases [1]. A schema is precise about structure and silent about meaning: it records that a column holds an integer, not that the integer represents a severity grade ranging from 0 to 5. A column named BM_DEP, which abbreviates *bone marrow depression*, is clear to the clinician who named it and opaque to everyone else.

An OWL ontology over the relational database schema closes that gap, supporting ontology-based data access over the live database [2], knowledge graph population [3], and semantic integration across heterogeneous sources [4]. Building one by hand does not scale to a schema with hundreds of columns. Automatic alternatives are cheap but shallow: the W3C Direct Mapping [5] turns each table into a class and each foreign key into an object property, guaranteeing that no column is lost while adding no class hierarchy, no disjointness, no readable labels, and no links to established vocabularies [6].

RIGOR [7] keeps the coverage guarantee and recovers the missing semantics. It walks the schema table by table in foreign-key order, starting each table from its deterministic direct mapping. A generative LLM enriches that direct mapping with context from the schema and its documentation, from external domain ontologies, from the core ontology built so far, and from the LLM's parametric knowledge, to produce an ontology fragment corresponding to this table. Then, a Judge-LLM validates the ontology fragment before it is merged into the full ontology. Across 3 databases, RIGOR was ranked ahead of direct mapping and LLM baselines by an LLM-as-a-judge and by 10 human experts.

The people who own these relational databases, however, are rarely the people who can run RIGOR: it meant cloning a repository, editing configuration files, and reading Python. We present the *RIGOR App*, which removes that barrier by putting the full pipeline behind a single Web interface, where uploading a schema and downloading its ontology are the only steps required. The user uploads a schema, optionally uploads schema documentation, external vocabularies, and a seed ontology to grow,

ISWC 2026 Companion Volume, October 25–29, 2026, Bari, Italy
*Corresponding author.
nadeen.fathallah@ki.uni-stuttgart.de (N. Fathallah); steffen.staab@ki.uni-stuttgart.de (S. Staab)
0000-0001-7921-034X (N. Fathallah); 0000-0002-9177-0312 (M. Nayyeri); 0000-0002-0780-4154 (S. Staab)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

then downloads an OWL2DL ontology. Beyond packaging the method, the *RIGOR App* (i) lowers the entry cost from a Python environment to a single upload and broadens the input it accepts from SQL schemas to plain CSV files, inferring column types and flagging problems such as a missing primary key; and (ii) makes the output trustworthy by passing it through the validation and repair stack introduced in [8, 9], which checks syntax, repairs logical inconsistency with HermiT [10] and ROBOT [11], and detects Critical and Important modeling pitfalls with OOPS! [12]. Deterministic repairs are applied automatically, whereas repairs that could change domain meaning are presented to the user for review. This human-in-the-loop stage allows domain owners to approve individual axioms, propose alternatives, or retain intentional modeling choices. The app also (iii) degrades gracefully, returning a valid direct mapping when no LLM API key is supplied; and (iv) lets the user inspect the result as Turtle or as an interactive graph. We extend the evaluation of RIGOR beyond its original setting. We used the *RIGOR App* to generate ontologies for the Northwind relational database, once with DeepSeek-V3 and once with Claude Opus 4.6. Before repair, OOPS! identified 17 Important findings in the DeepSeek-V3 ontology and 5 in Claude Opus 4.6. The *RIGOR App* automatically repaired 14 and 4, respectively; the rest required user input for semantic decisions.

2. Related Work

Work on deriving an ontology from a relational database differs in where it expects to find the semantics of the schema: the meaning its tables and columns carry but do not state.

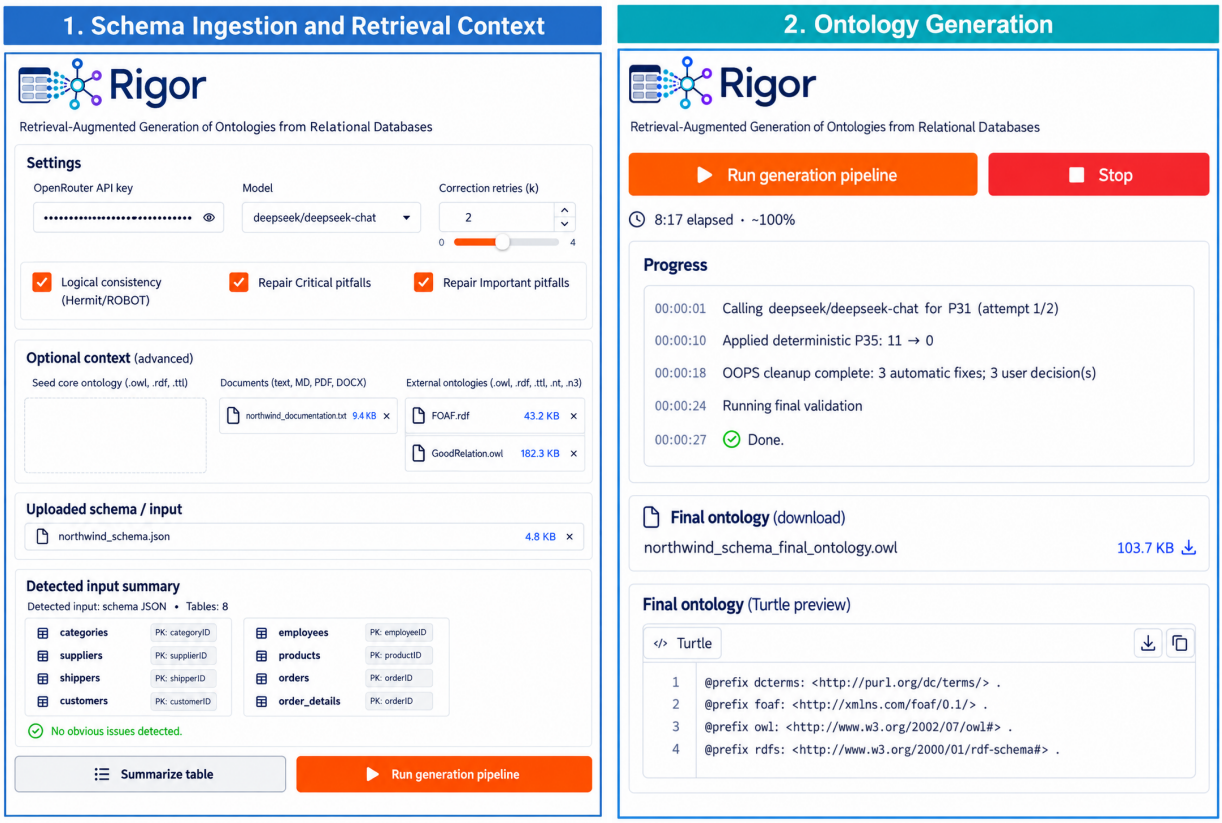
Rule-based and declarative mapping methods derive semantic representations from schema structure. The W3C Direct Mapping [5] deterministically maps tables, columns, and foreign keys to RDF terms, while R2RML [13] and D2RQ [14] provide mappings from relational data to RDF. These approaches are predictable but do not infer semantics that are absent from the schema or from supplied mappings.

Alignment-based methods borrow an existing ontology. BootOX [15] applies direct schema mapping, then runs an ontology matcher to align the resulting classes and properties with a domain ontology supplied by the user. Karma [16] and IncMap [17] map schema elements onto a target vocabulary. All three yield links to established terminology, and none can produce a target ontology when one does not already exist. The RODI benchmark [18] further showed that these systems, along with MIRROR [19] and D2RQ, mismap or skip schemas whose structure diverges from their meaning, such as tables that encode a hierarchy implicitly or junction tables that represent relationships rather than entities.

Schema-driven enrichment methods infer additional axioms from database structure and constraints. Ben Mahria et al. [6] apply predefined transformation rules to keys, constraints, and schema patterns to generate a richer ontology. These rules go beyond direct schema translation but remain limited to anticipated patterns and assume a well-structured input schema.

LLM-based methods use language models to construct ontologies from unstructured text or database schemas. Ontology learning from text [8, 9, 20, 21] lacks a relational schema to ground generation and therefore cannot account for foreign keys, junction tables, or denormalized designs. LLM4VKG [22] uses database schemas to generate virtual knowledge graph mappings, but requires a predefined target ontology and operates in a single pass, allowing early errors to propagate. RIGOR [7] takes as input a relational database schema and, optionally, schema documentation, relevant external ontologies, and an existing seed ontology to grow, and outputs a schema-grounded OWL ontology. It traverses the relational database tables in foreign-key order. For each table, it first applies a deterministic direct mapping to produce an ontology fragment that represents the table, its columns, keys, and foreign keys, thereby ensuring schema coverage. A Gen-LLM then enriches this ontology fragment using its pretrained knowledge and, when available, context retrieved from the schema documentation, external ontologies, the seed ontology, and the ontology built so far. A Judge-LLM validates the enriched ontology fragment before merging it into the ontology built so far. The final ontology includes labels, hierarchies, relationships, vocabulary alignments, and provenance without requiring a predefined target ontology.

Existing tools support related tasks but not the full workflow. Ontop [2] uses fixed rules to generate an ontology from a relational database schema. Karma maps a relational database to an ontology



(a) Schema and context. (b) Generation and preview.

Figure 1: RIGOR App workflow for schema ingestion, retrieval context, ontology generation, and preview.

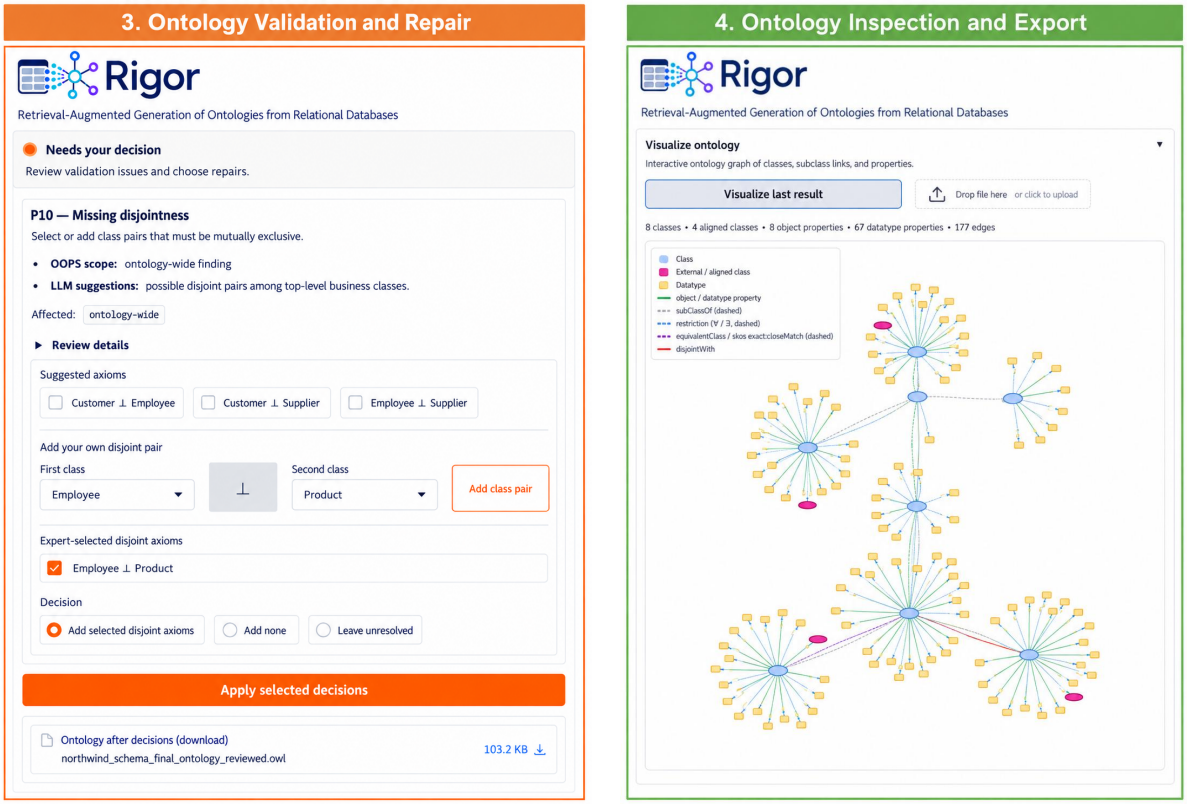
provided by the user. D2RQ [14] applies fixed rules to generate an RDF vocabulary and mapping from the relational schema, allowing the database to be queried as a virtual RDF graph. To our knowledge, no public tool can take a relational schema without a target ontology and produce a semantically enriched OWL2DL ontology with meaningful labels, class hierarchies, vocabulary alignments, and provenance. The *RIGOR App* fills this gap.

3. System

The *RIGOR App* operationalizes the RIGOR method [7] through a single-page Gradio interface. It accepts a relational schema in JSON or CSV format, with optional schema documentation, relevant external ontologies, and a seed ontology, and runs the table-wise Gen-LLM/Judge-LLM workflow described in Section 2. It outputs a validated OWL2DL ontology, a Turtle preview, an interactive graph, and, when semantic decisions are required, a decision report. It can also validate and repair an existing RDF/OWL ontology. Everything runs locally except calls to the language model and the OOPS! service. Below, we describe the schema input it accepts (Section 3.1), the optional context that drives retrieval (Section 3.2), how generation is exposed and configured (Section 3.3), the validation applied to the result (Section 3.4), and how the ontology is inspected and exported (Section 3.5).

3.1. Schema Ingestion

Only one input is required. It may be an SQL schema in JSON, which RIGOR consumes directly, or a plain CSV file, in which case the *RIGOR App* infers a type for each column, guesses a primary key, and synthesizes a single-table schema. Before running anything, the user can press *Summarize table*, which reports the tables found, including their columns, primary keys, and foreign key counts, along with



(a) Validation and repair.

(b) Inspection and export.

Figure 2: RIGOR App workflow for ontology validation and repair, followed by graph-based inspection and export.

warnings for missing primary keys, empty columns, or duplicate column names. The schema-upload and summary interface is shown in Figure 1a.

3.2. Retrieval Context

Three optional inputs correspond to RIGOR's retrieval sources. *Documents*, provided as text, Markdown, PDF, or Word files, supply descriptions that clarify cryptic schema elements such as `BM_DEP`. *External ontologies* provide established vocabularies for alignment, while a *seed core ontology* allows the pipeline to extend an existing ontology rather than start from scratch.

At the start of a run, the app uses the lightweight `all-MiniLM-L6-v2` sentence transformer [23] to index documents and external ontologies separately in FAISS, and reports the number of indexed chunks. `all-MiniLM-L6-v2` has a low computational cost, making it suitable for interactive retrieval. Omitting any retrieval input disables only the corresponding retrieval source; the rest of the pipeline proceeds unchanged. Figure 1a shows these optional inputs.

3.3. Generation

Before running, the user sets three things. The first is an OpenRouter API key, held in memory for the session and never written to disk. The second is the model: OpenRouter routes to a broad catalog of open and closed-source models across free and paid tiers, and any of them can be entered, so the app is not tied to a vendor. The third is the correction budget k. Recall that each generated ontology fragment is checked by the Judge-LLM before it is merged; when the Judge rejects an ontology fragment, it is sent back to the Gen-LLM for another attempt, and k caps how many such attempts are made before the pipeline gives up on enrichment for that table and keeps its direct mapping instead. A larger k trades runtime for a better chance of a clean ontology fragment. Pressing *Run* starts the pipeline, which

proceeds table by table and streams its progress with an elapsed timer, naming each step to the user as it happens: the schema is loaded and cleaned, the sentence-embedding model used for retrieval is initialized, and the *documents*, the *external ontologies*, and the *seed core ontology* are each embedded into a FAISS index. The tables are then visited in foreign-key order. For each table, the app builds its direct mapping, retrieves the most relevant documents, external concepts, and previously built growing ontology, and passes them to the Gen-LLM, which returns an enriched ontology fragment. The Judge-LLM checks the ontology fragment, the correction loop runs if needed, and the accepted ontology fragment is merged into the growing ontology before the next table is visited. Without an OpenRouter API key, the app runs no language model at all: it returns the direct mapping, the mechanical schema-to-OWL translation described in Section 2, with none of the labels, hierarchy, or alignments that the model would otherwise add. The streamed generation process and ontology preview are shown in Figure 1b.

3.4. Validation and Repair

An ontology assembled from language-model output can contain errors that are difficult to detect by inspection. The *RIGOR App* therefore passes the generated ontology through the validation and repair stack introduced in NeOn-GPT [8, 9]. Syntax errors are detected with `rdflib` and repaired automatically. Logical inconsistency is checked with HermiT [10] through ROBOT [11] and repaired automatically. Critical and Important modeling pitfalls are detected with OOPS! [12]. Modeling pitfalls are automatically addressed using an unambiguous, semantics-preserving solution. When a repair could change the intended domain meaning, the *RIGOR App* creates a decision card for the user (Figure 2a). The card identifies the pitfall, shows the affected ontology elements and supporting evidence, and presents only pitfall-specific actions, so the user can choose how to resolve it. For example, in the case of missing disjointness, the user may approve any subset of the suggested class pairs, select additional classes from ontology-populated dropdowns, reject all proposed axioms, or leave the finding unresolved. After the user presses *Apply selected decisions*, the chosen axioms are added or removed from a copy of the ontology. Each change is committed only if the result remains syntactically valid and logically consistent, preserves the schema entities, and introduces no new Critical or Important findings. Rejected changes are reported to the user with their validation reason. The *RIGOR App* then exposes a separate reviewed ontology for download, along with a decision-validation report that records which repairs were applied. The reasoning stages require Java and the HermiT and ROBOT jars, which the app locates automatically or accepts as paths. If they are unavailable, or if the OOPS! service cannot be reached, the corresponding stage is skipped with an explicit message, and the ontology is left unchanged.

3.5. Inspection and Export

The final ontology is available for download in OWL format. It can also be previewed as Turtle (Figure 1b) or rendered as an interactive graph (Figure 2b), which reports how many classes, properties, restrictions, and external alignments the run produced, and distinguishes classes aligned to an external vocabulary from locally minted ones. The interactive graph exports as a standalone HTML file.

4. Demonstration

We demonstrate and evaluate the *RIGOR App* on the eight-table Northwind database [24]. The input consists of the database schema in JSON, schema documentation in TXT, and two external vocabularies: GoodRelations [25, 26] and FOAF [27]. Before generation, *Summarize table* verifies the detected tables, columns, primary keys, and foreign keys. We run the *RIGOR App* twice, once with DeepSeek-V3 and once with Claude Opus 4.6 (Figure 1a). For both runs, we use the same Judge-LLM retry budget of $k = 2$. Pressing *Run* streams the foreign-key-ordered workflow (Figure 1b). The app indexes 9 documentation chunks and 337 external-ontology chunks and, for each table, retrieves up to three items from each available context source. The Gen-LLM enriches each table's direct mapping with the retrieved context;

the Judge-LLM validates and, when needed, corrects the fragment before it is merged. The completed ontology then undergoes syntax and consistency checks, and modeling-pitfall validation and repair, with semantic ambiguities presented to the user (Figure 2a). After final validation, the ontology can be inspected as an interactive graph (Figure 2b).

5. Results

Table 1 summarizes the ontologies generated by *RIGOR App* in the two Northwind relational database runs described in Section 4. Both runs preserve complete schema coverage and include provenance links to source tables or columns. Claude Opus 4.6 produces the larger ontology. DeepSeek-V3 contains 17 Important OOPS! findings before repair, of which 14 are fixed automatically; Claude Opus 4.6 contains 5, of which 4 are fixed automatically. The remaining findings for P10, P24, and P30 require semantic user decisions. This separation is deliberate: technical defects are automated, whereas domain-semantic choices remain under user control. The generated ontologies also contain semantic enrichment beyond direct mapping. For example, `order_details.quantity` receives a label, description, alignment to `gr:amountOfThisGood`, and a `prov:wasDerivedFrom` link. Classes such as Employees and Products are aligned with `foaf:Person` and `gr:ProductOrServiceModel`, respectively. The Judge-LLM additionally corrects fragments during generation. In the Claude Opus 4.6 run, the Gen-LLM proposed aligning the `homePhone` data property beneath `foaf:phone`, an object property; the Judge-LLM flagged the type mismatch and corrected the fragment before merging into the growing ontology.

Table 1
RIGOR App results for the Northwind relational database. Cov. is structural coverage of source tables and columns; Prov. counts `prov:wasDerivedFrom` assertions. Ext. Cls., Ext. Obj., and Ext. Data count externally aligned classes, object properties, and data properties. Auto counts Important OOPS! findings repaired without user input; User counts findings surfaced for semantic review.

Model	Cls.	Obj.	Data	Triples	Cov.	Prov.	Ext. Cls.	Ext. Obj.	Ext. Data	Auto	User
DeepSeek-V3	8	8	67	1,210	100%	83	6	1	6	14	3
Claude Opus 4.6	9	11	67	1,599	100%	86	4	0	3	4	1

6. Conclusion

The *RIGOR App* turns a schema-to-ontology RIGOR method into a tool its intended users can easily run: a database owner uploads a schema, optionally adds documentation and vocabularies, and downloads a sound OWL2DL ontology, with no code and no configuration. It makes RIGOR's pipeline accessible through a single interface, automatically repairs 14 of 17 and 4 of 5 Important pitfalls across the two Northwind relational database runs, and supports validated human decisions for the remaining semantic choices without sacrificing schema coverage.

Declaration on Generative AI

During the preparation of this work, the authors used Claude Opus 4.6 and DeepSeek-V3 within the described pipeline for ontology generation and validation, and Grammarly for grammar and spelling checks. After using these tools, the authors reviewed and edited the content as needed and took full responsibility for the publication's content.

References

[1] D.-E. Spanos, P. Stavrou, N. Mitrou, Bringing relational databases into the semantic web: A survey, Semantic Web 3 (2012) 169–209. URL: https://www.semanticscholar.org/paper/dd009177c67ac57be2c91c166d6b40cf5b21669a.

[2] G. Xiao, D. Lanti, R. Kontchakov, S. Komla-Ebri, E. Güzel-Kalaycı, L. Ding, J. Corman, B. Cogrel, D. Calvanese, E. Botoeva, The virtual knowledge graph system ontop, in: International Semantic Web Conference, Springer, 2020, pp. 259–277.

[3] A. Iglesias-Molina, D. Van Assche, J. Arenas-Guerrero, B. De Meester, C. Debruyne, S. Jozashoori, P. Maria, F. Michel, D. Chaves-Fraga, A. Dimou, The rml ontology: A community-driven modular redesign after a decade of experience in mapping heterogeneous data to rdf, in: International Semantic Web Conference, Springer, 2023, pp. 152–175.

[4] H. Wache, T. Vögele, U. Visser, H. Stuckenschmidt, G. Schuster, H. Neumann, S. Hübner, Ontology-based integration of information - A survey of existing approaches, in: A. Gómez-Pérez, M. Gruninger, H. Stuckenschmidt, M. Uschold (Eds.), Proceedings of the IJCAI-01 Workshop on Ontologies and Information Sharing Seattle, USA, August 4-5, 2001, volume 47 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2001. URL: https://ceur-ws.org/Vol-47/wache.pdf.

[5] J. F. Sequeda, M. Arenas, D. P. Miranker, On directly mapping relational databases to rdf and owl, in: Proceedings of the 21st international conference on World Wide Web, 2012, pp. 649–658. URL: https://arxiv.org/abs/1202.3667v3.

[6] B. Ben Mahria, I. Chaker, A. Zahi, A novel approach for learning ontology from relational database: From the construction to the evaluation, Journal of Big Data 8 (2021) 25. URL: https://www.semanticscholar.org/paper/A-novel-approach-for-learning-ontology-from-from-to-Mahria-Chaker/0b4573911a36be67367b1e5197a52a329682c2f1.

[7] N. Fathallah, M. Nayyeri, A. A. Yogi, R. B. Thapa, H.-M. Tautenhahn, A. Schnurpel, S. Staab, Retrieval-augmented generation of ontologies from relational databases, 2026. Accepted for publication at ISWC 2026. Preprint: https://arxiv.org/abs/2506.01232v1.

[8] N. Fathallah, A. Das, S. De Giorgis, A. Poltronieri, P. Haase, L. Kovriguina, NeOn-GPT: A large language model-powered pipeline for ontology learning, in: European Semantic Web Conference (ESWC), Springer, 2024, pp. 36–50. URL: https://www.semanticscholar.org/paper/15998546d59c8554e090fcc956f85c50164ac043.

[9] N. Fathallah, A. Das, S. De Giorgis, A. Poltronieri, P. Haase, L. Kovriguina, A. Meroño-Peñuela, E. Simperl, S. Staab, A. Algergawy, Extended neon-gpt: Advancing llm-powered ontology learning through ontology reuse and automated verification, Semantic Web 17 (2026) 22104968261453138.

[10] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, Hermit: an owl 2 reasoner, Journal of automated reasoning 53 (2014) 245–269. URL: https://www.semanticscholar.org/paper/cde4ef129340d21aec4519560db351e69abcd8bd.

[11] R. C. Jackson, J. P. Balhoff, E. Douglass, N. L. Harris, C. J. Mungall, J. A. Overton, Robot: a tool for automating ontology workflows, BMC bioinformatics 20 (2019) 407. URL: https://www.semanticscholar.org/paper/3e8ff7142d3c7c53dda71810551182a6247e29af.

[12] M. Poveda-Villalón, A. Gómez-Pérez, M. C. Suárez-Figueroa, OOPS! (ontology pitfall scanner!): An on-line tool for ontology evaluation, International Journal on Semantic Web and Information Systems (IJSWIS) 10 (2014) 7–34. doi:10.4018/IJSWIS.2014040102.

[13] W. W. W. Consortium, et al., R2rml: Rdb to rdf mapping language (2012).

[14] C. Bizer, A. Seaborne, D2rq-treating non-rdf databases as virtual rdf graphs, in: Proceedings of the 3rd international semantic web conference (ISWC2004), volume 2004, Proceedings of ISWC2004, Hiroshima, 2004. URL: https://www.semanticscholar.org/paper/930bb77b2b3fc705ff7940d8ce75d093154be042.

[15] E. Jiménez-Ruiz, E. Kharlamov, D. Zheleznyakov, I. Horrocks, C. Pinkel, M. G. Skjæveland, E. Thorstensen, J. Mora, Bootox: Practical mapping of rdbs to OWL 2, in: M. Arenas, Ó. Corcho, E. Simperl, M. Strohmaier, M. d'Aquin, K. Srinivas, P. Groth, M. Dumontier, J. Heflin,

K. Thirunarayan, S. Staab (Eds.), The Semantic Web - ISWC 2015 - 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II, volume 9367 of *Lecture Notes in Computer Science*, Springer, 2015, pp. 113–132. URL: https://doi.org/10.1007/978-3-319-25010-6_7. doi:10.1007/978-3-319-25010-6_7.

[16] C. A. Knoblock, P. Szekely, J. L. Ambite, A. Goel, S. Gupta, K. Lerman, M. Muslea, M. Taheriyan, P. Mallick, Semi-automatically mapping structured sources into the semantic web, in: Extended semantic web conference, Springer, 2012, pp. 375–390. URL: https://www.semanticscholar.org/paper/519a1530c6e2ecfe761950337bef29070c9656bc.

[17] C. Pinkel, C. Binnig, E. Jiménez-Ruiz, E. Kharlamov, A. Nikolov, A. Schwarte, C. Heupel, T. Kraska, Incmap: A journey towards ontology-based data integration, Lecture Notes in Informatics (LNI), Proceedings-Series of the Gesellschaft fur Informatik (GI) 265 (2017) 145–164.

[18] C. Pinkel, C. Binnig, E. Jiménez-Ruiz, E. Kharlamov, W. May, A. Nikolov, A. S. Bastinos, M. G. Skjæveland, A. Solimando, M. Taheriyan, C. Heupel, I. Horrocks, RODI: benchmarking relational-to-ontology mapping generation quality, Semantic Web 9 (2018) 25–52. URL: https://doi.org/10.3233/SW-170268. doi:10.3233/SW-170268.

[19] L. F. de Medeiros, F. Priyatna, O. Corcho, Mirror: Automatic r2rml mapping generation from relational databases, in: International conference on web engineering, Springer, 2015, pp. 326–343. URL: https://www.semanticscholar.org/paper/c81e290038b3ded9ebd33399270670ee96142ae8.

[20] A. Lo, A. Q. Jiang, W. Li, M. Jamnik, End-to-end ontology learning with large language models, in: A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/9e89f068a62f6858c661a8abecf5bb0a-Abstract-Conference.html.

[21] H. Babaei Giglou, J. D'Souza, S. Auer, Llms4ol: Large language models for ontology learning, in: International Semantic Web Conference, Springer, 2023, pp. 408–427. URL: https://arxiv.org/abs/2307.16648v2.

[22] G. Xiao, L. Ren, G. Qi, H. Xue, M. D. Panfilo, D. Lanti, LLM4VKG: leveraging large language models for virtual knowledge graph construction, in: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025, ijcai.org, 2025, pp. 4715–4723. URL: https://doi.org/10.24963/ijcai.2025/525. doi:10.24963/IJCAI.2025/525.

[23] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, M. Zhou, Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers, Advances in neural information processing systems 33 (2020) 5776–5788. URL: https://arxiv.org/abs/2002.10957v2.

[24] Microsoft, Northwind sample database, https://github.com/microsoft/sql-server-samples/tree/master/samples/databases/northwind-pubs, 2023. Accessed: 2026-07-16.

[25] M. Hepp, GoodRelations: The web vocabulary for e-commerce, http://purl.org/goodrelations/v1, 2011.

[26] M. Hepp, Goodrelations: An ontology for describing products and services offers on the web, in: International conference on knowledge engineering and knowledge management, Springer, 2008, pp. 329–346.

[27] D. Brickley, L. Miller, FOAF vocabulary specification 0.99, http://xmlns.com/foaf/spec/, 2014.